

Pac-Man Game Implementation Using Python

Submitted by: Shadrach Mulia Djohan | Student ID: 2902718931

School of Computing and Creative Arts, BINUS University

Course number: COMP6047001

Lecturer: Jude Joseph Lamug Martinez

Due Date: 16 January 2026

Table of Contents

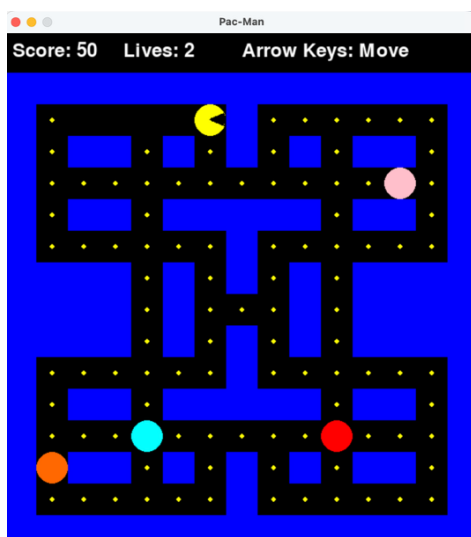
<i>Introduction</i>	<i>3</i>
<i>Game Design and Features.....</i>	<i>3</i>
<i>Design and Architecture</i>	<i>4</i>
Core Structure.....	4
OOP Design.....	4
Game States.....	4
Game Map Layout	5
Separation of Concerns.....	5
<i>Libraries and Modules</i>	<i>6</i>
pygame	6
Other Modules	6
<i>Algorithm and Logic</i>	<i>7</i>
Grid Traversal.....	7
Winning Algorithm.....	7
Ghost Algorithm	7
<i>Debugging and Testing</i>	<i>8</i>
Testing Table.....	8
Methodologies	8
<i>Statement on the Use of Artificial Intelligence</i>	<i>9</i>

Introduction

This project is a python-based 2D arcade game inspired by Atari's Pac-Man. The project is developed with the pygame library, providing features such as rendering graphics and handling user input. The purpose of the project is to implement fundamental algorithm principles such as variables, conditionals, loops, functions, and data structures in creating a real-world software application. The game is designed using the Object-Oriented Programming (OOP) principles to improve code readability and reusability.

Game Design and Features

The core gameplay revolves around Pac-Man consuming the pellets around the map. Pacman is controlled through the arrow keys on the keyboard. The game is won when all pellets are consumed with no less than one life remaining. The game is lost when the lives remaining has reached zero. Lives are initiated at two, decrementing by one for every instance Pac-Man collides with one of the ghosts. The lives system ensures that the game is not too challenging, avoiding over frustration from players. The ghosts serve as the main challenge of the game. They move around legal spaces of the map through a randomizing algorithm, creating obstacles at various parts of the map. Players are challenged to navigate the map through the least-risky route whilst having to consume all the pellets left.



Design and Architecture

Core Structure

The program is designed with object-oriented architecture and modularity in mind. This ensures that the code is readable, scalable, and maintainable. The core structure separates game logic, entity behaviour, and rendering which allows each section of the program to be developed and tested independently.

OOP Design

The program uses classes to separate the responsibility of each gameplay element, satisfying the encapsulation OOP principle. The “PacMan” class emulates player behaviour. It is initiated with the x and y values of the Pac-Man relative to the grid, its direction noted as a number, and a Boolean state on whether its mouth is opened. Within the class are functions called move and draw. These functions are specific to Pac-Man, hence why OOP is used in this game. On the other hand, the “Ghost” class emulates the enemy behaviour. It has similar states to the “PacMan” class, such as x and y values, as well as a unique state for its colour. The class promotes reusability as each ghost can be initiated through this class. The functions inside the class are identical to the Pac-Man class, implementing consistent design. This way, both classes share the same semantic meaning, whilst having different code implementations specific to their own needs, increasing readability.

Game States

While the “running” variable is initiated to True, the main game loop runs. Inside the loop, game states are manipulated. Game states either have the value “START,” “PLAYING,” “WIN,” or “GAME_OVER.” These states are fundamental to the game as they change which algorithm is ran for each state. Hence, game states are responsible in telling the main game

loop what to do next. They are created by design to improve readability, using capital letters to indicate a global variable. They prevent logic from overlapping and allow future expansion of the game through well-defined behaviours for each state.

Game Map Layout

The map of the game is created through a grid structure. This system simplifies the game design as all objects can be positioned relative to the grid. The game is initiated with a pre-defined map, “0” indicating a space with a pellet and “1” indicating walls. Every time Pac-Man occupies onto a 0 grid, the content of the grid is transformed to “2,” indicating a consumed pellet. This system creates a definable winning condition wherein no grid left has a value of “0.”

Separation of Concerns

The code is written to handle two main functionalities: logic and rendering. Logic specific to Pac-Man and the ghosts is self-contained as functions within their respective classes. Logic is running whenever the program is executed, hence they are all contained under the “while running” code block. Logic such as resetting the game is written as a function outside the code block, improving clarity and readability. Rendering the start, winning, and losing screen are contained as functions outside the main game loop. This is also done for the best clarity. Additionally, the rendering of the Pac-Man and Ghost characters are self-contained in each object, further separating general renders and character renders.

Libraries and Modules

pygame

pygame is the main library that acts as the backbone of the entire project. It allows the developer to render Python-based game logic into visual elements. Start, winning, and losing screens are created using fill to create a visual backdrop and render and blit to create texts and draw them respectively. pygame is also used for event handling through pygame.event. Keystrokes from the space bar and arrow keys serve as a 'continue' button and joystick respectively. Additionally, pygame allows movement to be synchronized through clock tick. This ensures that the timing for the movement of each element is equal, even across different devices.

Other Modules

1. sys: sys is a built-in module that provided system-specific parameters and functions that interact with the runtime environment. In my project sys.exit() is used to ensure that the Python interpreter stops its execution when the game is terminated.
2. random: the random module is used to randomize the ghosts' directions from a list of possible turns. This increases replayability of the game by making the ghosts' paths less predictable.
3. math: the math module is used to support the visual element of Pac-Man's mouth. Math functions to convert angles from degrees to radians which is required when using pygame's arc drawing function.

Algorithm and Logic

Grid Traversal

The grid traversal algorithm is set inside the “PacMan” and “Ghost” classes. In Pac-Man’s logic, arrow keystrokes are translated into a number that corresponds to a specific tuple in an array with predefined x and y values. A conditional checks whether the space is a wall or not (i.e., does not contain the value “1”). If the cell is legal, Pac-Man would occupy the space, and check whether a pellet exists. If it does, the Pac-Man would consume the pellet by changing the grid value to “2” from “0.”

Winning Algorithm

When the game’s state is set to “PLAYING,” the winning condition is checked whenever a set duration passes, and an arrow keystroke is registered. If “0” is not found anywhere within the grid, the game state would change to “WIN” which would prompt the win slide to appear.

Ghost Algorithm

For every ghost in the game, the losing algorithms would run by checking whether a ghost shares the same coordinates to Pac-Man. If True, the player’s life would be decremented by one. Then, within the same logical block, the algorithm would check whether the player has at least one life. If True, Pac-Man and the ghosts would all return to their original positions. Else, the game state would change to “GAME_OVER” which would prompt the game over slide to appear. The life-checking conditional is purposely placed only when a ghost shares the same coordinates to Pac-Man. This is because running the logic at other times are redundant as there is only one condition on losing a life.

Debugging and Testing

Testing Table

No.	Test Case	Prerequisite	Expected Result	Actual Result	Status
1	Win Condition	All pellets are consumed.	Win screen appears.	Win screen is displayed.	PASS
2	Lose Condition	Life is less than one.	Lose screen appears.	Lose screen is displayed.	PASS
3	Losing a Life	Pac-Man collides with a ghost.	Life is reduced by one.	Life is decremented by one.	PASS
4	Consuming a Pellet	Pac-Man moves to a grid with a pellet.	Pellet disappears and score increases.	Pellet is removed and score is updated.	PASS
5	Running onto a wall	Pac-Man runs onto a wall grid.	Pac-Man stays in place.	Pac-Man does not move.	PASS
6	Ghost Movement	Ghosts have legal paths to move.	Ghosts chooses valid directions.	Ghosts move through legal routes.	PASS

Methodologies

In the development phase, debugging is primarily done through observation and incremental fixes. Errors are rectified through tracing control flows and isolating dependent logic with independent components. Print statements are sometimes used to ensure logical flow is correct. Additionally, edge cases are tested, such as ghost movements and running into a wall, ensuring the consistency and correctness of the code before having the software application finalized.

Statement on the Use of Artificial Intelligence

Artificial Intelligence (AI) is used in the development in this game. The primary tool used is ChatGPT (OpenAI, GPT-4 model). AI is used in the planning, clarifying, and refinement phases of the project. All AI outputs have been reviewed and modified. Below are some examples of AI prompts used in this project:

- Brainstorm ideas for extending the complexity of the Pac-Man game.
- Why is the game state not changing when I press the space bar?
- How can I explain grid traversal and win conditions in an academic report?

No AI tools were used in creating substantial codebases without human revision. In using AI in my project, it has increased my knowledge in the subject, specifically on the use of the pygame module. AI has helped me practice in optimizing code, creating high-level feature ideas, and debugging.